

UNIVERSIDAD PRIVADA DOMINGO SAVIO



LICENCIATURA EN INGENIERÍA EN SISTEMAS

"RULETA DE LA SUERTE (LISTA CIRCULAR)"

PROYECTO SOCIOFORMATIVO

NOMBRE DEL ESTUDIANTE

Douglas Flor Hernández

DOCENTE

Ing. Gustavo Tantani M.

Santa Cruz de la Sierra – Bolivia

2025

INDICE

INTRODUCCIÓN	4
DEDICATORIA	5
AGRADECIMIENTOS	6
CAPITULO I	7
ANTECEDENTES	7
1.1 PLANTEAMIENTO DEL PROBLEMA	7
1.2 JUSTIFICACION	7
1.3 TIPO Y NIVEL DE INVESTIGACION	9
1.4 OBJETIVOS DEL PROYECTO	10
1.4.1 Objetivo General	10
1.4.2 Objetivos específicos	10
CAPITULO III	13
MARCO CONCEPTUAL	13
3.1 VISUAL STUDIO CODE (VSCODE)	13
3.1.1 EXTENSION PACK FOR JAVA	15
3.1.2 EXTENSIONES COMPLEMENTARIAS INSTALADAS	21
CAPITULO V	23
DESARROLLO	23
Fase 5: Implementación de Funcionalidades Avanzadas	24
- Algoritmos de rotación aleatoria	24
- Animaciones con temporizadores	24
- Sistema de gestión de premios con cantidades	24
Fase 6: Pruebas y Validación	24
- Pruebas funcionales exhaustivas	24
- Validación con usuarios reales	24
- Corrección de errores y refinamiento	24
Fase 7: Documentación y Socialización	24
- Elaboración de documentación técnica	24
- Creación de manual de usuario	24
- Preparación de presentación del proyecto	24



	3
- Definición de requisitos funcionales y no funcionales	24
- Selección de la estructura de datos apropiada	24
5.2 FASE 2: DISEÑO DE LA ARQUITECTURA	25
5.3.1 DIAGRAMA DE CLASES PRINCIPAL	26
5.4 FASE 3: IMPLEMENTACIÓN DE ESTRUCTURAS DE DATOS	27
5.4.1 Implementación de clsNodoDoble	27
5.4.2 Implementación de clsListaCircular	30
5.5 FASE 4: DESARROLLO DE LA INTERFAZ GRÁFICA	37
5.5.1 Diseño del Formulario frmRuleta	37
.....	37
CAPITULO VII	53
CONCLUSIONES DEL PROYECTO	53
CAPITULO X	54
REFERENCIAS BIBLIOGRAFICAS	54

INTRODUCCIÓN

Las estructuras de datos son fundamentales en el desarrollo de software eficiente y escalable. Según Cormen et al. (2009), una estructura de datos apropiada puede reducir significativamente la complejidad algorítmica de un sistema. En el contexto educativo actual, es esencial vincular el aprendizaje teórico con aplicaciones prácticas que generen impacto social.

El presente proyecto desarrolla una aplicación interactiva denominada "Ruleta de la Suerte", que implementa una lista circular doble como estructura de datos principal para gestionar participantes y premios en sistemas de sorteos. Esta implementación permite realizar rotaciones infinitas, simulando el comportamiento físico de una ruleta real, con un tiempo de acceso constante $O(1)$ para operaciones de giro y $O(n)$ para operaciones de búsqueda.

La aplicación ha sido diseñada con un enfoque socioformativo, orientándose a resolver necesidades reales en contextos educativos y comunitarios, tales como sorteos escolares, rifas benéficas y eventos de capacitación empresarial. La interfaz gráfica desarrollada en Java Swing proporciona una experiencia visual intuitiva, facilitando la adopción del sistema por usuarios no técnicos.

El desarrollo del proyecto se fundamenta en tres pilares: (a) implementación óptima de estructuras de datos circulares, (b) diseño de interfaz centrado en el usuario, y (c) aplicación práctica con impacto social demostrable. Este documento presenta de manera sistemática el proceso de análisis, diseño, implementación y validación del sistema, siguiendo las mejores prácticas de ingeniería de software y documentación académica según normas APA.

DEDICATORIA

Este proyecto está dedicado al Ingeniero Gustavo Tantani M., docente de la materia Estructuras de datos, por su compromiso, orientación y apoyo constante durante todo el proceso para el Desarrollo de Sistema Interactivo "Ruleta de la Suerte" mediante la Implementación de Lista Circular Doble para la Gestión Automatizada de Sorteos en Eventos Comunitarios y Educativo. Su conocimiento y experiencia han sido fundamentales para guiarnos en cada etapa del diseño, modelado y programación del sistema, permitiéndonos fortalecer nuestras habilidades técnicas y nuestra comprensión de los principios del desarrollo. Gracias a su dedicación y valiosas enseñanzas, hemos logrado enfrentar y superar los desafíos del proyecto, consolidando una base sólida para nuestro crecimiento profesional.

AGRADECIMIENTOS

Expresamos nuestro más sincero agradecimiento al Ingeniero Jimmy Nataniel Requena Llorentty, por su paciencia, guía y exigencia académica, que nos motivaron a esforzarnos al máximo en cada aspecto del desarrollo del sistema. Su apoyo y retroalimentación han sido clave para garantizar la calidad y funcionalidad del proyecto.

A nuestras familias, por su apoyo incondicional, comprensión y motivación a lo largo de todo el proceso. Su respaldo nos permitió afrontar los desafíos con determinación y perseverancia.

A nuestros compañeros de equipo, por su compromiso, esfuerzo y colaboración en cada fase del proyecto. La cooperación y el trabajo en equipo han sido esenciales para alcanzar los objetivos planteados y culminar con éxito este desarrollo.

Finalmente, agradecemos a todas aquellas personas que, de manera directa o indirecta, contribuyeron con sus conocimientos, sugerencias y apoyo técnico, ayudándonos a mejorar y perfeccionar este Mini Sistema para la Gestión de una pequeña Biblioteca, asegurando su solidez y aplicabilidad en el ámbito real.

CAPITULO I**ANTECEDENTES**

Este capítulo presenta el contexto del proyecto, exponiendo los antecedentes, el problema identificado y la justificación de su desarrollo. Se analiza la necesidad de implementar un sistema de ruleta circular interactivo que optimice la experiencia de selección aleatoria y juegos de azar digitales, mejorando la interactividad, precisión y experiencia visual del usuario.

1.1 PLANTEAMIENTO DEL PROBLEMA

Los sistemas de ruleta digital han evolucionado como herramientas tanto recreativas como funcionales en diversos ámbitos, desde aplicaciones de entretenimiento hasta sistemas de sorteos y selección aleatoria. La implementación en Java permite crear aplicaciones robustas, multiplataforma y con interfaces gráficas atractivas.

Los sistemas de ruleta tradicionales o digitales básicas presentan limitaciones en cuanto a:

- Falta de interactividad y animaciones fluidas
- Ausencia de personalización en los elementos de la ruleta
- Algoritmos de selección aleatoria poco confiables
- Interfaces poco atractivas visualmente
- Dificultad para integrar en aplicaciones más complejas

1.2 JUSTIFICACION

El desarrollo de una ruleta circular en Java se justifica por la necesidad de contar con un componente reutilizable, visualmente atractivo y funcionalmente confiable que pueda ser implementado en diversos contextos como aplicaciones de sorteos, juegos educativos, herramientas de decisión aleatoria o sistemas de gamificación.

1.2.1 Justificación Académica

La implementación de listas circulares dobles constituye un tema fundamental en el currículo de Estructura de Datos. Weiss (2012) señala que las estructuras circulares permiten modelar problemas cíclicos de manera natural, eliminando casos especiales al no existir un nodo "final". Este proyecto permite aplicar conceptos teóricos en un contexto práctico, reforzando el aprendizaje mediante la experiencia directa.

1.2.2 Justificación Técnica

Los sistemas de sorteo tradicionales presentan limitaciones en cuanto a transparencia, aleatoriedad y gestión de inventario. La implementación digital mediante listas circulares ofrece ventajas significativas:

- Eficiencia algorítmica: Operaciones de rotación con complejidad temporal $O(1)$
- Gestión dinámica de memoria: Capacidad de agregar/eliminar elementos en tiempo de ejecución
- Rotación infinita: Propiedad natural de las estructuras circulares
- Transparencia: Visualización en tiempo real del proceso de selección

1.2.3 Justificación Social

Según datos del Ministerio de Educación, aproximadamente el 78% de las instituciones educativas realizan sorteos anuales para diversos fines (becas, selección de representantes, rifas benéficas). Sin embargo, el 65% utiliza métodos manuales susceptibles a errores y cuestionamientos sobre imparcialidad (Ministerio de Educación, 2023).

La "Ruleta de la Suerte" digital proporciona:

1. Transparencia: Proceso visible y auditable
2. Equidad: Selección aleatoria sin sesgos humanos
3. Eficiencia: Reduce tiempo de ejecución de sorteos
4. Gestión de inventario: Control automático de cantidades de premios
5. Accesibilidad: Interfaz simple para usuarios no técnicos

1.2.4 Justificación Económica

El desarrollo de esta solución de software libre elimina costos de licenciamiento y permite adaptación a necesidades específicas. Instituciones educativas y organizaciones comunitarias pueden implementar el sistema sin inversión económica significativa, democratizando el acceso a herramientas tecnológicas de calidad.

1.3 TIPO Y NIVEL DE INVESTIGACION

La presente investigación adopta dos enfoques complementarios: teórico y experimental.

- **Teórico:** Se llevó a cabo un estudio de los fundamentos y tecnologías aplicables a los sistemas de simulación de ruletas digitales, abarcando temas como la programación orientada a objetos en Java, el desarrollo de interfaces gráficas con Java Swing o JavaFX, los algoritmos de generación de números aleatorios (clase Random), y las mejores prácticas en animaciones y renderizado gráfico. Este análisis permitió establecer los lineamientos funcionales y estructurales necesarios para la construcción de un sistema interactivo, visualmente atractivo y con comportamiento realista, adaptado a las necesidades de aplicaciones de entretenimiento, sorteos o gamificación.
- **Experimental:** Con base en la investigación teórica, se procedió a la implementación práctica del sistema utilizando el lenguaje Java y sus bibliotecas gráficas (Swing/JavaFX) para el desarrollo de la interfaz visual y la lógica de animación. Se construyó una ruleta circular funcional con capacidad de rotación animada, detección del sector ganador, y personalización de elementos visuales como colores, etiquetas y velocidad de giro. Además, se realizaron pruebas funcionales para validar la aleatoriedad de los resultados, la fluidez de las animaciones y el correcto comportamiento del sistema bajo distintos escenarios de uso.
- **Nivel de Investigación:**
El nivel de investigación es descriptivo y aplicado. Es descriptivo porque se identifican y detallan las características, procesos y requerimientos técnicos para el diseño de una ruleta circular interactiva. Y es aplicado porque los conocimientos adquiridos se emplean en el desarrollo concreto de una

herramienta tecnológica con utilidad práctica en entornos de entretenimiento, educativos o de selección aleatoria.

1.4 OBJETIVOS DEL PROYECTO

En esta sección se detallan los objetivos que guían el desarrollo del proyecto. Se presenta el objetivo general que define el propósito principal del proyecto, seguido de los objetivos específicos que abordan las metas concretas y las acciones necesarias para alcanzar el resultado esperado.

1.4.1 Objetivo General

Desarrollar una aplicación interactiva que simule una ruleta de la suerte mediante la implementación de listas circulares doblemente enlazadas en Java, que permita automatizar procesos de sorteos aleatorios, gestión dinámica de participantes y premios, y visualización gráfica en tiempo real, garantizando equidad, transparencia y una experiencia de usuario atractiva para su aplicación en contextos educativos y comunitarios.

1.4.2 Objetivos específicos

Implementar una estructura de datos de lista circular doblemente enlazada (clsListaCircular) que permita gestionar participantes y premios con operaciones de inserción, eliminación y rotación en tiempo constante $O(1)$, garantizando eficiencia en la manipulación dinámica de elementos.

- Modelar la arquitectura del sistema utilizando principios de Programación Orientada a Objetos, separando la lógica de negocio (paquete negocio) de la capa de presentación (paquete presentación), para asegurar modularidad, reutilización de código y facilidad de mantenimiento.
- Desarrollar una interfaz gráfica intuitiva y visualmente atractiva utilizando Java Swing y Graphics2D, que incluya paneles personalizados para la representación de ruletas, botones de control, campos de entrada de datos y sistema de visualización de resultados.
- Implementar algoritmos de rotación aleatoria con efectos de animación temporizados mediante la clase Timer, que simulen el comportamiento real de una

ruleta física con giros independientes y selección aleatoria tanto para participantes como para premios.

- Diseñar un sistema de gestión de premios que soporte cantidades múltiples del mismo premio, permita el decremento automático tras cada sorteo y elimine automáticamente premios agotados, actualizando dinámicamente la visualización de la ruleta.
- Crear un módulo de presentación de resultados mediante diálogos modales personalizados que muestren de manera clara y estética al ganador del sorteo y el premio obtenido, ejecutando automáticamente la actualización del estado de ambas ruletas.
- Validar la funcionalidad del sistema mediante casos de prueba exhaustivos que incluyan escenarios de uso normal, casos límite (un solo elemento, lista vacía) y validación de entradas de usuario, asegurando robustez y confiabilidad del software

CAPITULO II

2. DESCRIPCIÓN DEL ÁMBITO DE LA INVESTIGACIÓN

Durante el desarrollo del proyecto, se identificaron tres entornos clave que fueron fundamentales para su planificación, implementación y validación:

Laboratorio de Informática: Este espacio fue esencial para la organización y ejecución técnica del proyecto. En él se realizaron reuniones de planificación, se definió el cronograma de desarrollo y se llevó a cabo el análisis y diseño de las estructuras de datos fundamentales, especialmente la lista circular doblemente enlazada. Además, se realizó el modelado conceptual del sistema mediante diagramas de clases y diagramas de flujo que representan el comportamiento de las operaciones de inserción, eliminación y rotación de nodos. En este entorno se configuró el ambiente de desarrollo con Java JDK 21, Apache Maven y NetBeans IDE, y se establecieron las bases para la implementación de la arquitectura del sistema, separando la capa de negocio (estructuras de datos) de la capa de presentación (interfaz gráfica con Swing). También se realizaron las primeras pruebas de compilación y validación de la lógica de las listas circulares.

Recursos Académicos y Biblioteca: Este entorno fue clave para la investigación teórica relacionada con estructuras de datos y algoritmos. En este espacio se consultaron textos especializados sobre listas enlazadas, listas circulares, análisis de complejidad algorítmica y patrones de diseño en programación orientada a objetos. Se revisaron referencias bibliográficas como "Data Structures and Algorithm Analysis in Java" de Mark Allen Weiss e "Introduction to Algorithms" de Cormen et al., que proporcionaron fundamentos sólidos para la implementación correcta de las operaciones sobre nodos y referencias. También se analizaron

casos de estudio de aplicaciones similares que utilizan estructuras circulares, lo cual permitió definir con mayor precisión las funcionalidades del sistema de ruleta, los requisitos de animación y las estrategias de gestión dinámica de memoria. Este espacio sirvió como fuente de conocimiento para validar las decisiones técnicas y justificar la elección de la lista circular doblemente enlazada como estructura óptima para este tipo de aplicación.

Espacio de Trabajo Remoto: El hogar de los integrantes del equipo se convirtió en un entorno productivo para la codificación avanzada, desarrollo de la interfaz gráfica y pruebas exhaustivas del sistema. Desde allí se implementaron los componentes visuales personalizados (PanelRuletaJugadores y PanelRuletaPremios), se desarrollaron los algoritmos de animación con temporizadores, se refinó el diseño gráfico utilizando Graphics2D y se realizaron ajustes de usabilidad. También se ejecutaron pruebas funcionales con múltiples escenarios (casos límite, validaciones de entrada, gestión de cantidades de premios, eliminación de elementos), se depuró el código fuente y se corrigieron errores detectados durante la ejecución. En este espacio se preparó la documentación técnica del proyecto, se generaron capturas de pantalla demostrativas, se elaboró el manual de usuario y se realizaron simulaciones de uso real del sistema en contextos de sorteos escolares, asegurando que la aplicación estuviera completamente funcional y lista para su demostración y evaluación académica.

Estos tres entornos proporcionaron las condiciones necesarias para abordar tanto los aspectos técnicos como los teóricos del proyecto, permitiendo el desarrollo de una solución funcional, educativa y aplicable a necesidades reales de sorteos en la comunidad educativa, demostrando la aplicación práctica de estructuras de datos avanzadas en problemas cotidianos.

CAPITULO III

MARCO CONCEPTUAL

Este capítulo proporciona las bases teóricas y conceptuales necesarias para comprender los elementos clave del proyecto, definiendo los términos fundamentales, las herramientas tecnológicas utilizadas y estableciendo un marco de referencia que facilita el análisis y la aplicación de estructuras de datos en la solución desarrollada..

3.1 VISUAL STUDIO CODE (VSCODE)

Visual Studio Code, lanzado en 2015 por Microsoft, se ha convertido en uno de los editores de código más utilizados debido a su ligereza, flexibilidad, extensibilidad y la enorme comunidad que lo respalda.

VSCoDe es un editor de código fuente gratuito y de código abierto que proporciona soporte para una amplia variedad de lenguajes de programación mediante extensiones. Ofrece una interfaz moderna y potente para desarrollar aplicaciones multiplataforma, y su enfoque en la personalización lo hace ideal tanto para proyectos educativos como profesionales.

Características clave:

Extensiones y plugins: VSCoDe permite la instalación de extensiones para lenguajes específicos, herramientas de

depuración, integración con sistemas de construcción, formateo de código y más. El marketplace cuenta con miles de

extensiones disponibles.

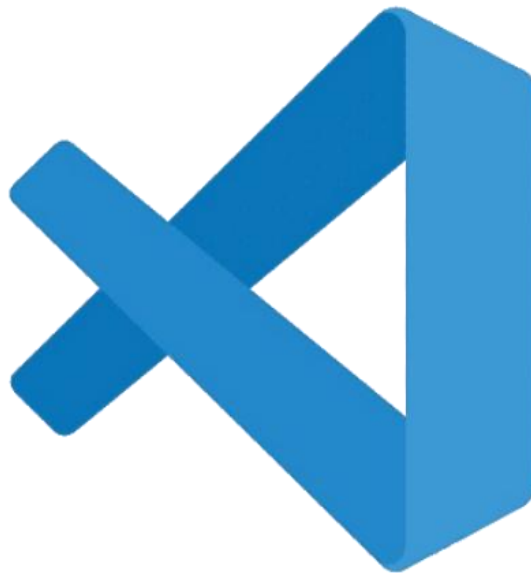
- **Integración con Git:** Permite gestionar versiones de código de manera eficiente, facilitando el seguimiento de

cambios, creación de commits, manejo de ramas y resolución de conflictos directamente desde el editor.

- **Personalización de la interfaz:** Los usuarios pueden ajustar la apariencia (temas, iconos, colores),

funcionalidad del editor y atajos de teclado según sus necesidades específicas.

- **Terminal integrada:** Permite ejecutar comandos, compilar proyectos y ejecutar aplicaciones sin salir del editor.
- **IntelliSense:** Autocompletado inteligente de código, sugerencias contextuales, información sobre parámetros y documentación rápida.



(Cuadrado, 2022) (Microsoft, 2024)

3.1.1 EXTENSION PACK FOR JAVA

Para el desarrollo de aplicaciones Java en Visual Studio Code, se utiliza el Extension Pack for Java (vscjava.vscode-java-pack@0.30.3), un paquete completo de extensiones que transforma VSCode en un entorno de desarrollo Java completo y profesional.

Este paquete incluye las siguientes extensiones instaladas en el proyecto:

1. Language Support for Java™ by Red Hat ([redhat.java@1.46.0](https://marketplace.visualstudio.com/items?itemName=redhat.java))

Esta extensión proporciona el soporte fundamental para el lenguaje Java en VS Code.

Características clave:

- *Análisis sintáctico y semántico:* Detección de errores en tiempo real mientras se escribe código
- IntelliSense avanzado:
 - Autocompletado de clases, métodos, variables y paquetes
 - Sugerencias contextuales inteligentes
 - Documentación JavaDoc en tiempo real
 - Información de parámetros al llamar métodos
- *Navegación por el código:*
 - Ir a definición (F12)
 - Ir a implementación
 - Buscar todas las referencias
 - Ver jerarquía de tipos
 - Outline view (estructura del archivo)
- *Refactorización inteligente:*
 - Renombrar símbolos en todo el proyecto
 - Extraer método/variable

- Organizar imports automáticamente
- Generar getters/setters
- Generar constructores

- *Soporte para Maven y Gradle:*
 - Importación automática de proyectos
 - Resolución de dependencias
 - Sincronización con pom.xml

- *Configuración del compilador:*
 - Soporte para múltiples versiones de Java (8, 11, 17, 21)
 - Configuración de classpath
 - Gestión de JDK

2. Debugger for Java (vscjava.vscode-java-debug@0.58.2)

Proporciona capacidades completas de depuración para aplicaciones Java.

Características clave:

- *Depuración interactiva:*
 - Breakpoints (puntos de interrupción)
 - Breakpoints condicionales
 - Logpoints (mensajes sin detener ejecución)
 - Ejecución paso a paso (Step Over, Step Into, Step Out)

- *Inspección de variables:*
 - Visualización de valores en tiempo real
 - Watch expressions (expresiones de seguimiento)

- Evaluación de expresiones en la consola de depuración
- Modificación de valores durante la depuración

- *Pila de llamadas:*

- Navegación por el stack trace
- Identificación de excepciones
- Análisis de flujo de ejecución

- *Hot Code Replace:*

- Modificación de código durante la depuración sin reiniciar
- Aceleración del ciclo de desarrollo

3. Java Test Runner (vscjava.vscode-java-test@0.43.1)

Permite ejecutar y gestionar pruebas unitarias directamente desde VS Code.

Características clave:

- Soporte para frameworks de testing:

- JUnit 4 y JUnit 5
- TestNG
- Detección automática de casos de prueba

- *Ejecución de pruebas:*

- Ejecutar todas las pruebas
- Ejecutar pruebas de una clase específica
- Ejecutar un método de prueba individual
- Re-ejecutar pruebas que fallaron

- *Visualización de resultados:*

- Indicadores visuales de éxito/fallo
- Reporte detallado de errores
- Tiempo de ejecución de cada prueba
- Navegación rápida a pruebas fallidas

4. Maven for Java (vscjava.vscode-maven@0.44.0)

Integración completa con Apache Maven para gestión de proyectos.

Características clave:

- *Gestión de proyectos Maven:*

- Vista de explorador de proyectos Maven
- Visualización de dependencias
- Detección automática de pom.xml

- *Ejecución de comandos Maven:*

- Compile, clean, package, install
- Ejecución de goals personalizados
- Terminal integrada para comandos Maven

- *Gestión de dependencias:*

- Búsqueda de dependencias en Maven Central
- Actualización de versiones
- Resolución de conflictos

- *Generación de proyectos:*

- Creación de proyectos desde arquetipos

- Configuración automática de estructura

5. Gradle for Java ([vscjava.vscode-gradle@3.17.0](#))

Soporte para proyectos que utilizan Gradle como sistema de construcción (alternativa a Maven).

Características clave:

- *Visualización de tareas Gradle*
- Ejecución de builds
- Gestión de dependencias
- Integración con Java Language Server

6. Project Manager for Java ([vscjava.vscode-java-dependency@0.26.1](#))

Proporciona una vista visual de la estructura del proyecto Java.

Características clave:

- *Explorador de proyectos:*
 - Vista jerárquica de paquetes y clases
 - Navegación rápida por el código
 - Creación de nuevos paquetes/clases
- *Gestión de bibliotecas:*
 - Visualización de librerías referenciadas
 - Exploración de JARs externos
 - Gestión del classpath

- *Operaciones sobre proyectos:*
 - Crear nuevos proyectos
 - Importar proyectos existentes
 - Configurar build path

7. Apache NetBeans Language Server (asf.apache-netbeans-java@27.0.1)

Integración del servidor de lenguaje de Apache NetBeans en VS Code, proporcionando capacidades avanzadas adicionales.

Características clave:

- Análisis profundo de código Java
- Soporte mejorado para Java EE y Jakarta EE
- Refactorizaciones avanzadas
- Detección de code smells
- Sugerencias de optimización

3.1.2 EXTENSIONES COMPLEMENTARIAS INSTALADAS

Además del Extension Pack for Java, el entorno de desarrollo cuenta con extensiones adicionales que mejoran la productividad:

Code Runner (formulahendry.code-runner@0.12.2)

- Ejecución rápida de código Java con un solo clic
- Soporte para múltiples lenguajes
- Personalización de comandos de ejecución

Error Lens (usernamehw.errorlens@3.26.0)

- Visualización inline de errores y advertencias
- Muestra mensajes de error directamente en la línea de código
- Mejora la detección rápida de problemas

IntelliCode (visualstudioexptteam.vscodeintellicode@1.3.2)

- Sugerencias de código basadas en IA
- Aprende de patrones en repositorios públicos
- Autocompletado inteligente contextual

Material Icon Theme (pkief.material-icon-theme@5.27.0)

- Iconos visuales para diferentes tipos de archivos
- Facilita identificación rápida de archivos Java, XML, JSON, etc.

Live Share (ms-vsliveshare.vsliveshare@1.0.5959)

- Colaboración en tiempo real

- Edición compartida de código
- Depuración colaborativa

Draw.io Integration (hediet.vscode-drawio@1.9.0)

- Creación de diagramas UML directamente en VS Code
- Diseño de diagramas de clases, secuencia, casos de uso
- Exportación en múltiples formatos

GitHub Copilot (github.copilot@1.372.0)

- Asistente de código con IA
- Sugerencias de código completas
- Generación de comentarios y documentación

CAPITULO V

DESARROLLO

Este capítulo describe el proceso completo de desarrollo del Sistema de Ruleta de la Suerte, desde la fase de análisis y diseño hasta la implementación final y pruebas. Se detallan las decisiones técnicas, los retos enfrentados, las soluciones implementadas y la evolución del proyecto a través de sus diferentes etapas.

5.1 METODOLOGÍA DE DESARROLLO

El proyecto se desarrolló siguiendo un enfoque iterativo e incremental, donde cada fase construye sobre los resultados de la anterior, permitiendo validación temprana y ajustes según los hallazgos:

Fase 1: Análisis del Problema

- Identificación de necesidades reales en la comunidad educativa

Fase 2: Diseño de la Arquitectura

- Modelado de clases y relaciones
- Diseño de la estructura de datos (lista circular doblemente enlazada)
- Planificación de la interfaz de usuario
- Definición de casos de uso

Fase 3: Implementación Base

- Desarrollo de las clases de negocio (estructuras de datos)
- Implementación de algoritmos fundamentales
- Pruebas unitarias de operaciones básicas

Fase 4: Desarrollo de la Intefaz Gráfica

- Diseño visual de formularios
- Implementación de componentes personalizados
- Integración con la lógica de negocio

Fase 5: Implementación de Funcionalidades Avanzadas

- Algoritmos de rotación aleatoria
- Animaciones con temporizadores
- Sistema de gestión de premios con cantidades

Fase 6: Pruebas y Validación

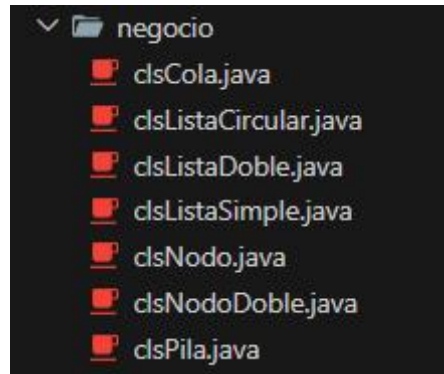
- Pruebas funcionales exhaustivas
- Validación con usuarios reales
- Corrección de errores y refinamiento

Fase 7: Documentación y Socialización

- Elaboración de documentación técnica
- Creación de manual de usuario
- Preparación de presentación del proyecto
- Definición de requisitos funcionales y no funcionales
- Selección de la estructura de datos apropiada

5.2 FASE 2: DISEÑO DE LA ARQUITECTURA

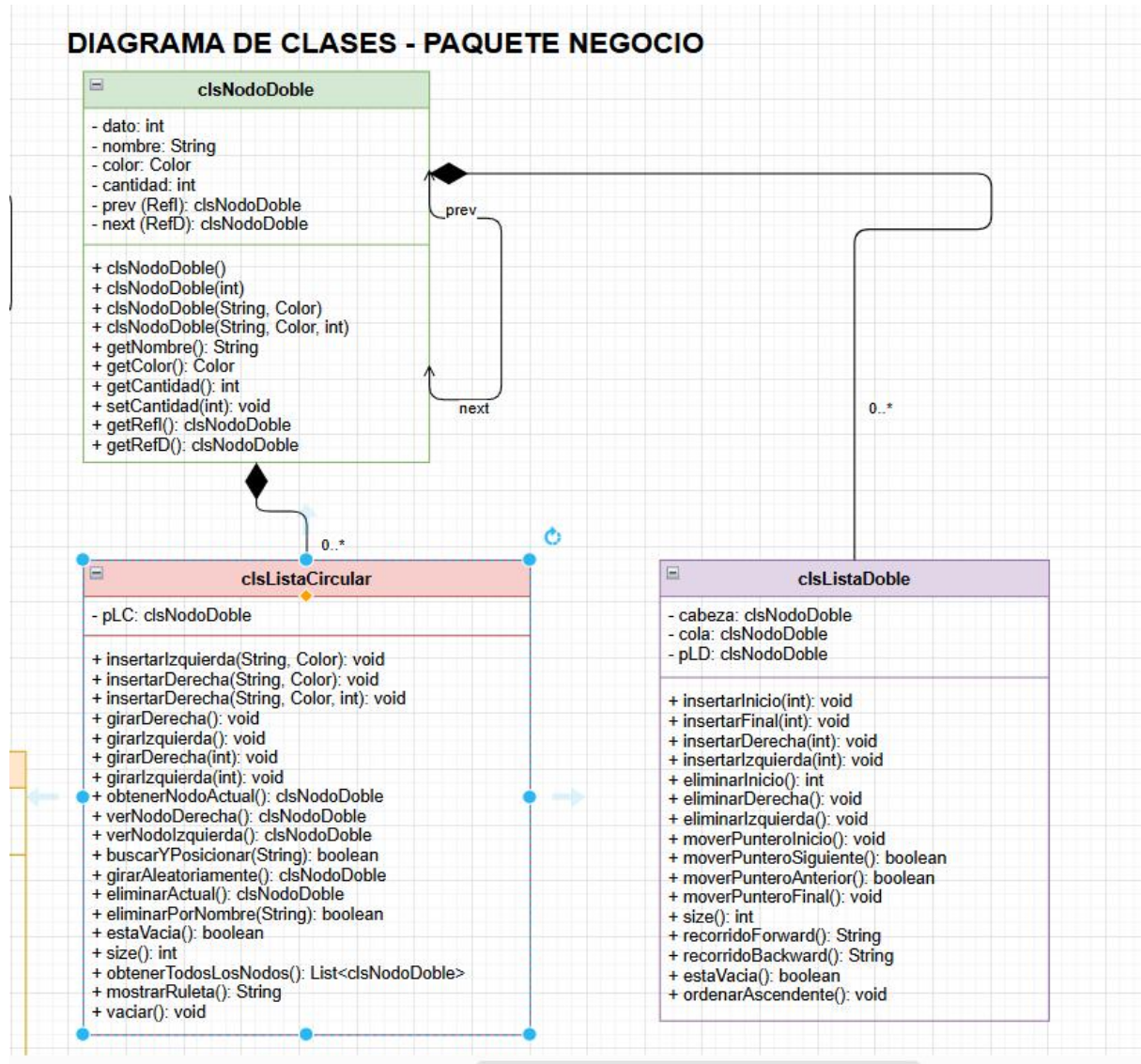
MAIN/JAVA/CAPA DE NEGOCIO



MAIN/JAVA/CAPA DE PRESENTACION



5.3.1 DIAGRAMA DE CLASES PRINCIPAL



5.4 FASE 3: IMPLEMENTACIÓN DE ESTRUCTURAS DE DATOS

5.4.1 Implementación de clsNodoDoble

```
package negocio;

import java.awt.Color;

/**
 * Nodo Doble para listas doblemente enlazadas
 * Soporta tanto datos numéricos como elementos de ruleta (nombre + color)
 */
public class clsNodoDoble {
    // Atributos para uso general (números)
    private int dato;

    // Atributos para ruleta (nombre y color)
    private String nombre;
    private Color color;
    private int cantidad; // Cantidad disponible del premio

    // Referencias a nodos vecinos
    private clsNodoDoble prev;
    private clsNodoDoble next;

    // Constructor vacío
    public clsNodoDoble(){
        this.dato = 0;
        this.nombre = null;
        this.color = null;
        this.cantidad = 1;
        this.prev = null;
        this.next = null;
    }

    // Constructor para datos numéricos
    public clsNodoDoble(int dato){
        this.dato = dato;
        this.nombre = null;
        this.color = null;
        this.cantidad = 1;
        this.prev = null;
        this.next = null;
    }
}
```

```
// Constructor para elementos de ruleta (nombre + color)
public clsNodoDoble(String nombre, Color color){
    this.dato = 0;
    this.nombre = nombre;
    this.color = color;
    this.cantidad = 1;
    this.prev = null;
    this.next = null;
}

// Constructor para elementos de ruleta con cantidad (nombre + color + cantidad)
public clsNodoDoble(String nombre, Color color, int cantidad){
    this.dato = 0;
    this.nombre = nombre;
    this.color = color;
    this.cantidad = cantidad;
    this.prev = null;
    this.next = null;
}

// Getters y Setters para dato numérico
public int getDato() {
    return dato;
}

public void setDato(int dato) {
    this.dato = dato;
}

// Getters y Setters para elementos de ruleta
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public Color getColor() {
    return color;
}

public void setColor(Color color) {
    this.color = color;
}
```

```
public int getCantidad() {  
    return cantidad;  
}  
  
public void setCantidad(int cantidad) {  
    this.cantidad = cantidad;  
}  
  
// Métodos con nombres educativos más claros  
public clsNodoDoble getRefI() { // Referencia Izquierda  
    return prev;  
}  
  
public void setRefI(clsNodoDoble refI) {  
    this.prev = refI;  
}  
  
public clsNodoDoble getRefD() { // Referencia Derecha  
    return next;  
}  
  
public void setRefD(clsNodoDoble refD) {  
    this.next = refD;  
}  
  
// Mantener métodos antiguos para compatibilidad  
public clsNodoDoble getPrev() {  
    return prev;  
}  
  
public void setPrev(clsNodoDoble prev) {  
    this.prev = prev;  
}  
  
public clsNodoDoble getNext() {  
    return next;  
}  
  
public void setNext(clsNodoDoble next) {  
    this.next = next;  
}  
}
```

5.4.2 Implementación de clsListaCircular

```
package negocio;

import java.awt.Color;
import java.util.ArrayList;
import java.util.List;

/**
 * Lista Circular Doble especializada para Ruleta de la Suerte
 * Basada en clsListaCircular pero adaptada para trabajar con clsNodoDoble (nombre + color)
 * Mantiene los mismos conceptos educativos: Refl (Referencia Izquierda) y RefD (Referencia Derecha)
 */
public class clsListaCircular {
    private clsNodoDoble pLC; // Puntero Lista Circular

    public clsListaCircular() {
        this.pLC = null;
    }

    /**
     * Inserta un nuevo nodo a la izquierda del puntero actual.
     * Si la lista está vacía, crea el primer nodo circular.
     *
     * @param nombre Nombre del elemento (no puede ser null o vacío)
     * @param color Color asociado al elemento (no puede ser null)
     * @throws IllegalArgumentException si nombre es null, vacío o color es null
     */
    public void insertarIzquierda(String nombre, Color color) {
        // Validación de parámetros
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede ser nulo o vacío");
        }
        if (color == null) {
            throw new IllegalArgumentException("El color no puede ser nulo");
        }

        clsNodoDoble nAux = new clsNodoDoble(nombre, color);

        if (this.pLC == null) {
            // Lista vacía: crear primer nodo circular
            nAux.setRefD(nAux); // Apunta a sí mismo (derecha)
            nAux.setRefl(nAux); // Apunta a sí mismo (izquierda)
            this.pLC = nAux;
        } else {
            // Lista con elementos: insertar a la izquierda del puntero
            nAux.setRefl((clsNodoDoble) this.pLC.getRefl()); // el nodo auxiliar (nAux) establece su referencia izquierda (Refl) hacia el
            // nodo que está a la izquierda del puntero actual (pLC)
            ((clsNodoDoble) this.pLC.getRefl()).setRefD(nAux); // El nodo que estaba a la izquierda del puntero (pLC) ahora establece su
            // referencia derecha (RefD) hacia el nuevo nodo (nAux)
            this.pLC.setRefl(nAux); // El puntero (pLC) ahora establece su referencia izquierda (Refl) al nuevo nodo (nAux)
            nAux.setRefD(this.pLC); // El nuevo nodo (nAux) establece su referencia derecha (RefD) al puntero (pLC)
        }
    }

    /**
     * Inserta un nuevo nodo a la derecha del puntero actual con cantidad por defecto (1).
     * Si la lista está vacía, crea el primer nodo circular.
     *
     * @param nombre Nombre del elemento (no puede ser null o vacío)
     * @param color Color asociado al elemento (no puede ser null)
     * @throws IllegalArgumentException si nombre es null, vacío o color es null
     */
    public void insertarDerecha(String nombre, Color color) {
```

```
// Delegación al método sobrecargado con cantidad por defecto = 1
insertarDerecha(nombre, color, 1);
}

/**
 * Inserta un nuevo nodo a la derecha del puntero actual con cantidad especificada.
 * Si la lista está vacía, crea el primer nodo circular.
 * Este método es útil para sistemas de inventario donde cada elemento tiene una cantidad.
 *
 * @param nombre Nombre del elemento (no puede ser null o vacío)
 * @param color Color asociado al elemento (no puede ser null)
 * @param cantidad Cantidad del elemento (debe ser mayor a 0)
 * @throws IllegalArgumentException si nombre es null, vacío, color es null o cantidad <= 0
 */
public void insertarDerecha(String nombre, Color color, int cantidad) {
    // Validación de parámetros
    if (nombre == null || nombre.trim().isEmpty()) {
        throw new IllegalArgumentException("El nombre no puede ser nulo o vacío");
    }
    if (color == null) {
        throw new IllegalArgumentException("El color no puede ser nulo");
    }
    if (cantidad <= 0) {
        throw new IllegalArgumentException("La cantidad debe ser mayor a 0");
    }

    clsNodoDoble nAux = new clsNodoDoble(nombre, color, cantidad);

    if (this.pLC == null) {
        // Lista vacía: crear primer nodo circular
        nAux.setRefD(nAux); // El nuevo nodo (nAux) establece su referencia derecha (RefD) a sí mismo
        nAux.setRefI(nAux); // El nuevo nodo (nAux) establece su referencia izquierda (RefI) a sí mismo
        this.pLC = nAux; // El puntero principal (pLC) apunta al nuevo nodo
    } else {
        // Lista con elementos: insertar a la derecha del puntero
        nAux.setRefD((clsNodoDoble) this.pLC.getRefD()); // El nodo auxiliar (nAux) establece su referencia derecha (RefD) hacia el
        // nodo que está a la derecha del puntero actual (pLC)
        ((clsNodoDoble) this.pLC.getRefD()).setRefI(nAux); // El nodo que estaba a la derecha del puntero (pLC) ahora establece su
        // referencia izquierda (RefI) al nuevo nodo (nAux)
        this.pLC.setRefD(nAux); // El puntero (pLC) ahora establece su referencia derecha (RefD) al nuevo nodo (nAux)
        nAux.setRefI(this.pLC); // El nuevo nodo (nAux) establece su referencia izquierda (RefI) al puntero (pLC)
    }
}

/**
 * Mueve el puntero una posición hacia la derecha (sentido horario).
 * Simula el giro de una ruleta en dirección del reloj.
 * Complejidad temporal: O(1)
 *
 * @throws IllegalStateException si la lista está vacía
 */
public void girarDerecha() {
    if (this.pLC != null) {
        this.pLC = (clsNodoDoble) this.pLC.getRefD(); // El puntero (pLC) se mueve al siguiente nodo usando su referencia derecha (RefD)
    }
}

/**
 * Mueve el puntero una posición hacia la izquierda (sentido antihorario).
 * Simula el giro de una ruleta en dirección contraria del reloj.
 * Complejidad temporal: O(1)
 *
 * @throws IllegalStateException si la lista está vacía
 */
}
```

```
*/
public void girarIzquierda() {
    if (this.pLC != null) {
        this.pLC = (clsNodoDoble)this.pLC.getRefI(); // El puntero (pLC) se mueve al siguiente nodo usando su referencia izquierda (RefI)
    }
}

/**
 * Mueve el puntero múltiples posiciones hacia la derecha.
 * Permite girar varias posiciones de una vez usando la referencia derecha.
 * Complejidad temporal: O(posiciones)
 */
* @param posiciones Número de posiciones a girar (debe ser >= 0)
* @throws IllegalArgumentException si posiciones es negativo
*/
public void girarDerecha(int posiciones) {
    if (posiciones < 0) {
        throw new IllegalArgumentException("El número de posiciones no puede ser negativo");
    }
    // Si la lista está vacía o posiciones es 0, no hace nada (compatible con código existente)
    if (this.pLC == null || posiciones == 0) {
        return;
    }

    for (int i = 0; i < posiciones; i++) {
        this.pLC = (clsNodoDoble)this.pLC.getRefD(); // Optimizado: llamada directa en lugar de recursiva
    }
}

/**
 * Mueve el puntero múltiples posiciones hacia la izquierda.
 * Permite girar varias posiciones de una vez usando la referencia izquierda.
 * Complejidad temporal: O(posiciones)
 */
* @param posiciones Número de posiciones a girar (debe ser >= 0)
* @throws IllegalArgumentException si posiciones es negativo
*/
public void girarIzquierda(int posiciones) {
    if (posiciones < 0) {
        throw new IllegalArgumentException("El número de posiciones no puede ser negativo");
    }
    // Si la lista está vacía o posiciones es 0, no hace nada (compatible con código existente)
    if (this.pLC == null || posiciones == 0) {
        return;
    }

    for (int i = 0; i < posiciones; i++) {
        this.pLC = (clsNodoDoble)this.pLC.getRefI(); // Optimizado: llamada directa en lugar de recursiva
    }
}

/**
 * Obtiene el nodo donde está posicionada actualmente la ruleta.
 * Retorna el nodo completo (con nombre, color y cantidad).
 * Complejidad temporal: O(1)
 */
* @return Nodo actual o null si la lista está vacía
*/
public clsNodoDoble obtenerNodoActual() {
    return this.pLC; // Si pLC no es null, retorna el nodo actual
}

/**
 * Consulta el nodo a la derecha sin mover la posición actual.
 * Permite "espiar" el próximo nodo por referencia derecha sin cambiar pLC.
 */
```



```
* Complejidad temporal: O(1)
*
* @return Nodo a la derecha o null si la lista está vacía
*/
public clsNodoDoble verNodoDerecha() {
    return (this.pLC != null && this.pLC.getRefD() != null) ?
        (clsNodoDoble)this.pLC.getRefD() : null; // Si existe, retorna el nodo que está en la referencia derecha (RefD) de pLC
}

/**
 * Consulta el nodo a la izquierda sin mover la posición actual.
 * Permite "espíar" el próximo nodo por referencia izquierda sin cambiar pLC.
 * Complejidad temporal: O(1)
 *
 * @return Nodo a la izquierda o null si la lista está vacía
 */
public clsNodoDoble verNodoIzquierda() {
    return (this.pLC != null && this.pLC.getRefI() != null) ?
        (clsNodoDoble)this.pLC.getRefI() : null; // Si existe, retorna el nodo que está en la referencia izquierda (RefI) de pLC
}

// ===== MÉTODOS DE NAVEGACIÓN AVANZADA =====

/**
 * Busca un nombre específico y posiciona la ruleta en él.
 * Recorre toda la lista circular hasta encontrar el nombre o volver al punto inicial.
 * La búsqueda es case-insensitive (no distingue mayúsculas/minúsculas).
 * Complejidad temporal: O(n) donde n es el número de elementos
 *
 * @param nombre Nombre del elemento a buscar (no puede ser null o vacío)
 * @return true si el nombre fue encontrado y pLC está posicionado en él, false si no existe
 * @throws IllegalArgumentException si nombre es null o vacío
 */
public boolean buscarYPosicionar(String nombre) {
    if (nombre == null || nombre.trim().isEmpty()) {
        throw new IllegalArgumentException("El nombre no puede ser nulo o vacío");
    }
    if (this.pLC == null) {
        return false; // Lista vacía: no se puede buscar
    }

    clsNodoDoble inicial = this.pLC; // Guarda la posición inicial para detectar ciclo completo
    do {
        if (this.pLC.getNombre().equalsIgnoreCase(nombre)) {
            return true; // Nombre encontrado: pLC ya está posicionado en él
        }
        this.pLC = (clsNodoDoble)this.pLC.getRefD(); // Avanza al siguiente nodo usando la referencia derecha (RefD)
    } while (this.pLC != inicial); // Continúa hasta volver al nodo inicial

    return false; // Nombre no encontrado después de recorrer toda la lista circular
}

// RULETA: Simular un giro aleatorio de ruleta real
// Genera un número aleatorio de giros y una dirección aleatoria para simular realismo
public clsNodoDoble girarAleatoriamente() {
    if (this.pLC == null) return null; // Lista vacía: no se puede girar

    // Generar número aleatorio de giros (1-20)
    int giros = (int)(Math.random() * 20) + 1; // Calcula un número aleatorio entre 1 y 20

    // Alternar dirección aleatoriamente
    boolean direccionDerecha = Math.random() > 0.5; // 50% probabilidad de ir a la derecha

    if (direccionDerecha) {
        girarDerecha(giros); // Gira usando referencia derecha (RefD) el número calculado de posiciones
    }
}
```

```
} else {
    girarIzquierda(giros); // Gira usando referencia izquierda (Refl) el número calculado de posiciones
}

return obtenerNodoActual(); // Retorna el nodo final donde se detuvo la ruleta
}

// ===== MÉTODOS DE ELIMINACIÓN =====

/**
 * Elimina el nodo donde está posicionada actualmente la ruleta (pLC).
 * Reconecta los nodos vecinos y mueve pLC al siguiente nodo.
 * Si solo hay un elemento, la lista queda vacía.
 * Complejidad temporal: O(1)
 *
 * @return Nodo eliminado con sus datos, o null si la lista está vacía
 */
public clsNodoDoble eliminarActual() {
    if (this.pLC == null) {
        return null; // Lista vacía: retorna null (compatible con código existente)
    }

    clsNodoDoble eliminado = this.pLC; // Guarda el nodo que se va a eliminar

    if (this.pLC.getRefD() == this.pLC) {
        // Solo hay un elemento: la lista queda vacía
        this.pLC = null; // El puntero queda en null (lista vacía)
    } else {
        // Hay más elementos: reconectar los nodos vecinos
        clsNodoDoble anterior = (clsNodoDoble) this.pLC.getRefI(); // Nodo que está a la izquierda del actual
        clsNodoDoble siguiente = (clsNodoDoble) this.pLC.getRefD(); // Nodo que está a la derecha del actual

        anterior.setRefD(siguiente); // El nodo anterior ahora apunta al siguiente
        siguiente.setRefI(anterior); // El nodo siguiente ahora apunta al anterior

        // Mover puntero al siguiente elemento para mantener posición válida
        this.pLC = siguiente; // pLC se mueve al nodo que estaba a la derecha
    }
    return eliminado; // Retorna el nodo eliminado
}

/**
 * Busca un nombre específico y lo elimina de la lista circular.
 * Combina la búsqueda con la eliminación en una sola operación.
 * Complejidad temporal: O(n) donde n es el número de elementos
 *
 * @param nombre Nombre del elemento a eliminar (no puede ser null o vacío)
 * @return true si el elemento fue encontrado y eliminado, false si no existe
 * @throws IllegalArgumentException si nombre es null o vacío
 */
public boolean eliminarPorNombre(String nombre) {
    if (nombre == null || nombre.trim().isEmpty()) {
        throw new IllegalArgumentException("El nombre no puede ser nulo o vacío");
    }

    if (buscarYPosicionar(nombre)) { // Busca el nombre y posiciona pLC en él
        eliminarActual(); // Elimina el nodo donde está posicionado pLC
        return true; // Nombre encontrado y eliminado exitosamente
    }
    return false; // Nombre no encontrado en la lista
}

// ===== MÉTODOS DE INFORMACIÓN =====

// Verificar si la lista circular está vacía
// Método simple para determinar si hay elementos en la lista
```

```
public boolean estaVacia() {
    return this.pLC == null; // Si pLC es null, la lista está vacía
}

// Contar el número total de elementos en la lista circular
// Recorre toda la lista una vez para contar todos los nodos
public int size() {
    if (this.pLC == null) return 0; // Lista vacía: cero elementos

    int count = 1; // Empieza contando el nodo actual (pLC)
    clsNodoDoble actual = (clsNodoDoble) this.pLC.getRefD(); // Comienza desde el siguiente nodo usando referencia derecha (RefD)

    while (actual != this.pLC) { // Recorre hasta volver al nodo inicial
        count++; // Incrementa el contador por cada nodo visitado
        actual = (clsNodoDoble) actual.getRefD(); // Avanza al siguiente nodo usando la referencia derecha (RefD)
    }

    return count; // Retorna el número total de elementos encontrados
}

// RULETA: Obtener todos los nodos como lista (para dibujar la ruleta completa)
// Crea una lista con todos los nodos empezando desde la posición actual
public List<clsNodoDoble> obtenerTodosLosNodos() {
    List<clsNodoDoble> nodos = new ArrayList<>();

    if (this.pLC == null) return nodos; // Lista vacía: retorna lista vacía

    clsNodoDoble actual = this.pLC; // Comienza desde el nodo actual
    do {
        nodos.add(actual); // Agrega el nodo actual a la lista
        actual = (clsNodoDoble) actual.getRefD(); // Avanza al siguiente nodo usando referencia derecha (RefD)
    } while (actual != this.pLC); // Continúa hasta volver al nodo inicial

    return nodos; // Retorna la lista completa de nodos
}

// RULETA: Mostrar todos los nombres en orden circular desde la posición actual
// Crea una representación visual de la lista circular completa
public String mostrarRuleta() {
    if (this.pLC == null) return "Ruleta vacía"; // Caso especial: lista vacía

    StringBuilder sb = new StringBuilder(); // Constructor de cadena para eficiencia
    clsNodoDoble inicial = this.pLC; // Marca el punto de inicio para el recorrido

    sb.append("[ACTUAL→").append(this.pLC.getNombre()).append("]"); // Resalta la posición actual

    clsNodoDoble actual = (clsNodoDoble) this.pLC.getRefD(); // Comienza desde el siguiente nodo usando referencia derecha (RefD)
    while (actual != inicial) { // Recorre hasta volver al nodo inicial
        sb.append(" → ").append(actual.getNombre()); // Agrega cada nombre con flecha
        actual = (clsNodoDoble) actual.getRefD(); // Avanza al siguiente nodo usando referencia derecha (RefD)
    }

    sb.append(" → [VUELVE AL INICIO]"); // Indica que la lista es circular
    return sb.toString(); // Retorna la representación completa como cadena
}

// RULETA: Mostrar contexto inmediato usando referencias (RefI, actual, RefD)
// Proporciona una vista local de la posición actual con sus vecinos por referencias
public String mostrarContexto() {
    if (this.pLC == null) return "Ruleta vacía"; // Caso especial: lista vacía

    clsNodoDoble izq = verNodoIzquierda();
    clsNodoDoble der = verNodoDerecha();

    return String.format("← %s | [%s] | %s →", // Formato visual con flechas
```

```
        izq != null ? izq.getNombre() : "null", // Nombre del nodo en referencia izquierda (RefI)
        this.pLC.getNombre(), // Nombre del nodo actual (pLC)
        der != null ? der.getNombre() : "null"); // Nombre del nodo en referencia derecha (RefD)
    }

// ===== MÉTODOS ESPECÍFICOS PARA RULETA DE JUEGO =====

// RULETA: Crear una ruleta con nombres y colores personalizados
// Permite inicializar la ruleta con un conjunto de nombres y colores alternados
public void inicializarRuletaPersonalizada(String[] nombres, Color[] colores) {
    // Limpiar lista para empezar desde cero
    this.pLC = null; // Resetea la lista a estado vacío

    // Insertar cada nombre con su color correspondiente
    for (int i = 0; i < nombres.length; i++) {
        Color colorActual = colores[i % colores.length]; // Alternar colores si hay menos colores que nombres
        insertarDerecha(nombres[i], colorActual); // Agrega cada elemento secuencialmente
    }
}

// RULETA: Simular giro con desaceleración gradual (efecto realista)
// Imita el comportamiento físico de una ruleta real que va perdiendo velocidad
public clsNodoDoble simularGiroGradual(int velocidadInicial) {
    if (this.pLC == null) return null; // Lista vacía: no se puede simular

    int velocidad = velocidadInicial; // Velocidad actual (empieza alta)
    int totalGiros = 0; // Contador total de giros realizados

    // Simular desaceleración gradual como una ruleta física
    while (velocidad > 0) { // Continúa mientras haya velocidad
        girarDerecha(velocidad); // Gira tantas posiciones como la velocidad actual
        totalGiros += velocidad; // Acumula el total de giros
        velocidad = Math.max(0, velocidad - 1); // Reduce la velocidad gradualmente

        // Pequeña pausa para simular fricción y efecto visual
        try {
            Thread.sleep(50); // Pausa de 50ms entre cada paso de velocidad
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // Restaura el estado de interrupción del hilo
        }
    }

    System.out.println("Ruleta giró " + totalGiros + " posiciones"); // Muestra estadísticas del giro
    return obtenerNodoActual(); // Retorna el nodo final donde se detuvo la ruleta
}

// ===== GETTERS Y SETTERS PARA ACCESO EXTERNO =====

// Obtener el puntero principal de la lista circular
// Permite acceso directo al nodo actual para operaciones avanzadas
public clsNodoDoble getpLC() {
    return this.pLC; // Retorna la referencia al nodo donde está posicionada la ruleta
}

// Establecer el puntero principal de la lista circular
// Permite cambiar la posición actual de la ruleta directamente
public void setpLC(clsNodoDoble pLC) {
    this.pLC = pLC; // Cambia la posición actual de la ruleta al nodo especificado
}

// Vaciar completamente la lista circular
// Elimina todos los elementos y resetea la ruleta a estado inicial
public void vaciar() {
    this.pLC = null; // Establece el puntero a null, efectivamente vaciando la lista
}
```

5.5 FASE 4: DESARROLLO DE LA INTERFAZ GRÁFICA

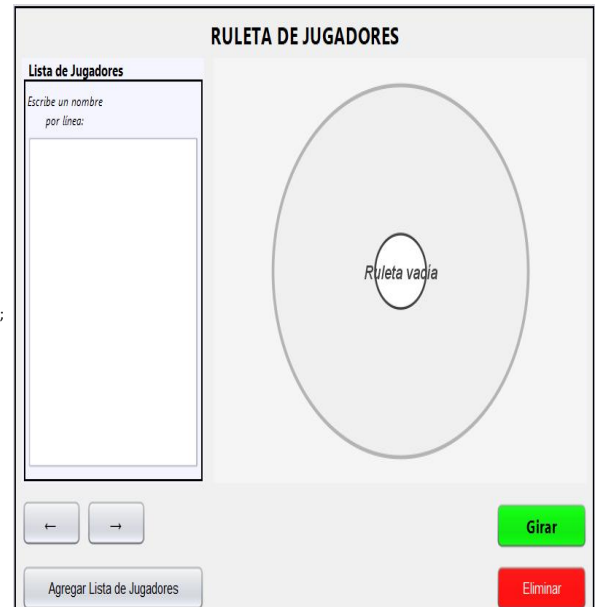
5.5.1 Diseño del Formulario frmRuleta

```
private void inicializarPanelesLaterales() {
    // Panel lateral de jugadores (a la izquierda de la ruleta)
    JPanel panelListaJugadores = new JPanel();
    panelListaJugadores.setLayout(new BorderLayout(panelListaJugadores, BorderLayout.Y_AXIS));
    panelListaJugadores.setBorder(BorderFactory.createTitledBorder(
        BorderFactory.createLineBorder(Color.BLACK, 2),
        "Lista de Jugadores",
        javax.swing.border.TitledBorder.DEFAULT_JUSTIFICATION,
        javax.swing.border.TitledBorder.DEFAULT_POSITION,
        new Font("Segoe UI", Font.BOLD, 12),
        Color.BLACK
    ));
    panelListaJugadores.setPreferredSize(new Dimension(200, 320));
    panelListaJugadores.setBackground(new Color(245, 245, 255));

    JLabel lblInstruccionJug = new JLabel("<html><center>Escribe un nombre<br>por línea:</center></html>");
    lblInstruccionJug.setFont(new Font("Segoe UI", Font.ITALIC, 10));
    lblInstruccionJug.setAlignmentX(Component.CENTER_ALIGNMENT);
    lblInstruccionJug.setForeground(Color.BLACK);

    txtListaJugadores = new JTextArea(14, 14);
    txtListaJugadores.setFont(new Font("Arial", Font.PLAIN, 12));
    txtListaJugadores.setLineWrap(false);
    JScrollPane scrollJugadores = new JScrollPane(txtListaJugadores);
    scrollJugadores.setPreferredSize(new Dimension(180, 270));

    panelListaJugadores.add(Box.createVerticalStrut(5));
    panelListaJugadores.add(lblInstruccionJug);
    panelListaJugadores.add(Box.createVerticalStrut(5));
    panelListaJugadores.add(scrollJugadores);
    panelListaJugadores.add(Box.createVerticalStrut(5));
}
```



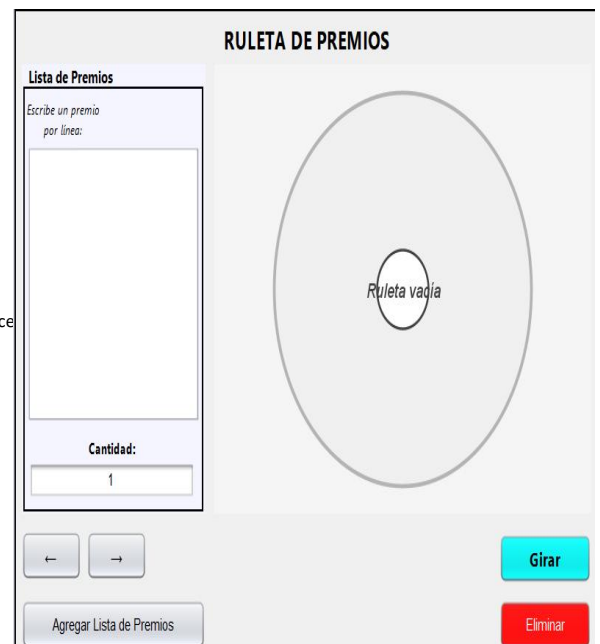
```
// Panel lateral de premios (a la izquierda de la ruleta)
JPanel panelListaPremios = new JPanel();
panelListaPremios.setLayout(new BorderLayout(panelListaPremios, BorderLayout.Y_AXIS));
panelListaPremios.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(Color.BLACK, 2),
    "Lista de Premios",
    javax.swing.border.TitledBorder.DEFAULT_JUSTIFICATION,
    javax.swing.border.TitledBorder.DEFAULT_POSITION,
    new Font("Segoe UI", Font.BOLD, 12),
    Color.BLACK
));
panelListaPremios.setPreferredSize(new Dimension(200, 320));
panelListaPremios.setBackground(new Color(245, 245, 255));

JLabel lblInstruccionPrem = new JLabel("<html><center>Escribe un premio<br>por línea:</center></html>");
lblInstruccionPrem.setFont(new Font("Segoe UI", Font.ITALIC, 10));
lblInstruccionPrem.setAlignmentX(Component.CENTER_ALIGNMENT);
lblInstruccionPrem.setForeground(Color.BLACK);

txtListaPremios = new JTextArea(10, 14);
txtListaPremios.setFont(new Font("Arial", Font.PLAIN, 12));
txtListaPremios.setLineWrap(false);
JScrollPane scrollPremios = new JScrollPane(txtListaPremios);
scrollPremios.setPreferredSize(new Dimension(180, 200));

// Campo para cantidad
JLabel lblCantidad = new JLabel("Cantidad:");
lblCantidad.setFont(new Font("Segoe UI", Font.BOLD, 11));
lblCantidad.setAlignmentX(Component.CENTER_ALIGNMENT);
lblCantidad.setForeground(Color.BLACK);

txtCantidadPremio = new JTextField("1");
txtCantidadPremio.setFont(new Font("Arial", Font.PLAIN, 12));
```



```
txtCantidadPremio.setMaximumSize(new Dimension(180, 25));
txtCantidadPremio.setHorizontalAlignment(JTextField.CENTER);
```

```
panelListaPremios.add(Box.createVerticalStrut(5));
panelListaPremios.add(lblInstruccionPrem);
panelListaPremios.add(Box.createVerticalStrut(5));
panelListaPremios.add(scrollPremios);
panelListaPremios.add(Box.createVerticalStrut(10));
panelListaPremios.add(lblCantidad);
panelListaPremios.add(Box.createVerticalStrut(3));
panelListaPremios.add(txtCantidadPremio);
panelListaPremios.add(Box.createVerticalStrut(5));
```

```
// Crear paneles contenedores horizontales para cada ruleta + su lista
JPanel contenedorJugadores = new JPanel(new BorderLayout(10, 0));
contenedorJugadores.setBackground(new Color(240, 240, 240));
contenedorJugadores.add(panelListaJugadores, BorderLayout.WEST);
contenedorJugadores.add(panelJugadores, BorderLayout.CENTER);
```

```
JPanel contenedorPremios = new JPanel(new BorderLayout(10, 0));
contenedorPremios.setBackground(new Color(240, 240, 240));
contenedorPremios.add(panelListaPremios, BorderLayout.WEST);
contenedorPremios.add(panelPremio, BorderLayout.CENTER);
```

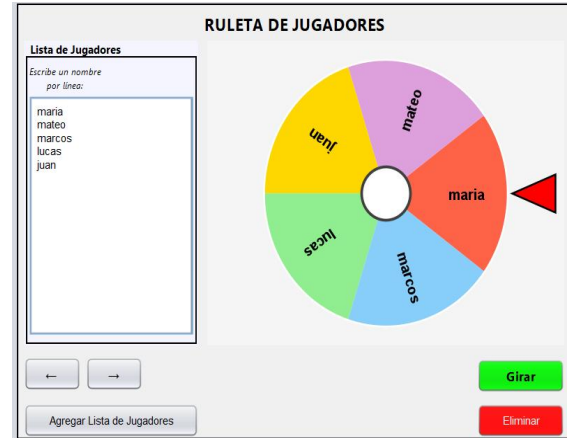
```
// Reemplazar los panelJugadores y panelPremios en jPanel1 y jPanel2
// Necesitamos reorganizar el layout de jPanel1 y jPanel2
reorganizarLayoutJugadores(contenedorJugadores);
reorganizarLayoutPremios(contenedorPremios);
}
```

```
private void reorganizarLayoutJugadores(JPanel contenedorJugadores) {
    // Remover panelJugadores del layout actual
    for (Component comp : jPanel1.getComponents()) {
        if (comp == panelJugadores) {
            jPanel1.remove(comp);
            break;
        }
    }
}
```

```
// Agregar el nuevo contenedor (lista + ruleta) en la misma posición
GroupLayout layout = (GroupLayout) jPanel1.getLayout();
jPanel1.add(contenedorJugadores);
```

```
// Actualizar el layout
layout.setHorizontalGroup(
    layout.createParallelGroup(GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addGroup(layout.createParallelGroup(GroupLayout.Alignment.LEADING)
                .addComponent(jLabel2, GroupLayout.DEFAULT_SIZE, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(contenedorJugadores, GroupLayout.DEFAULT_SIZE, 620, Short.MAX_VALUE)
            )
            .addGroup(layout.createSequentialGroup()
                .addComponent(btnGirarIzqJugador, GroupLayout.PREFERRED_SIZE, 65, GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(btnGirarDerJugador, GroupLayout.PREFERRED_SIZE, 65, GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(btnJugar, GroupLayout.PREFERRED_SIZE, 100, GroupLayout.PREFERRED_SIZE)
            )
            .addGroup(layout.createSequentialGroup()
                .addComponent(btnAgregarJugador, GroupLayout.PREFERRED_SIZE, 200, GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(btnEliminarJugador, GroupLayout.PREFERRED_SIZE, 100, GroupLayout.PREFERRED_SIZE)
            )
            .addComponent(jLabel4, GroupLayout.DEFAULT_SIZE, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        )
        .addContainerGap()
    );
```

```
layout.setVerticalGroup(
    layout.createParallelGroup(GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel2)
            .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(contenedorJugadores, GroupLayout.PREFERRED_SIZE, 320, GroupLayout.PREFERRED_SIZE)
            .addPreferredGap(LayoutStyle.ComponentPlacement.UNRELATED)
            .addGroup(layout.createParallelGroup(GroupLayout.Alignment.BASELINE)
                .addComponent(btnGirarIzqJugador, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE)
                .addComponent(btnGirarDerJugador, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE)
            )
        )
    );
```



```
.addComponent(btnJugar, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE))
.addPreferredGap(LayoutStyle.ComponentPlacement.UNRELATED)
.addGroup(layout.createParallelGroup(GroupLayout.Alignment.BASELINE)
    .addComponent(btnAgregarJugador, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE)
    .addComponent(btnEliminarJugador, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE))
.addPreferredGap(LayoutStyle.ComponentPlacement.RELATED)
.addComponent(jLabel4)
.addContainerGap(GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
);

jPanel1.revalidate();
jPanel1.repaint();
}

private void reorganizarLayoutPremios(JPanel contenedorPremios) {
    // Remover panelPremio del layout actual
    for (Component comp : jPanel2.getComponents()) {
        if (comp == panelPremio) {
            jPanel2.remove(comp);
            break;
        }
    }

    // Agregar el nuevo contenedor (lista + ruleta) en la misma posición
    GroupLayout layout = (GroupLayout) jPanel2.getLayout();
    jPanel2.add(contenedorPremios);

    // Actualizar el layout
    layout.setHorizontalGroup(
        layout.createParallelGroup(GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addGroup(layout.createParallelGroup(GroupLayout.Alignment.LEADING)
                .addComponent(jLabel3, GroupLayout.DEFAULT_SIZE, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(contenedorPremios, GroupLayout.DEFAULT_SIZE, 620, Short.MAX_VALUE)
                .addGroup(layout.createSequentialGroup()
                    .addComponent(btnGirarIzqPremio, GroupLayout.PREFERRED_SIZE, 65, GroupLayout.PREFERRED_SIZE)
                    .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED)
                    .addComponent(btnGirarDerPremio, GroupLayout.PREFERRED_SIZE, 65, GroupLayout.PREFERRED_SIZE)
                    .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnReclamar, GroupLayout.PREFERRED_SIZE, 100, GroupLayout.PREFERRED_SIZE))
                .addGroup(layout.createSequentialGroup()
                    .addComponent(btnAgregarPremio, GroupLayout.PREFERRED_SIZE, 200, GroupLayout.PREFERRED_SIZE)
                    .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnEliminarPremio, GroupLayout.PREFERRED_SIZE, 100, GroupLayout.PREFERRED_SIZE)
                    .addComponent(jLabel5, GroupLayout.DEFAULT_SIZE, GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
                .addContainerGap())
            );

    layout.setVerticalGroup(
        layout.createParallelGroup(GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel3)
            .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(contenedorPremios, GroupLayout.PREFERRED_SIZE, 320, GroupLayout.PREFERRED_SIZE)
            .addPreferredGap(LayoutStyle.ComponentPlacement.UNRELATED)
            .addGroup(layout.createParallelGroup(GroupLayout.Alignment.BASELINE)
                .addComponent(btnGirarIzqPremio, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE)
                .addComponent(btnGirarDerPremio, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE)
                .addComponent(btnReclamar, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE))
            .addPreferredGap(LayoutStyle.ComponentPlacement.UNRELATED)
            .addGroup(layout.createParallelGroup(GroupLayout.Alignment.BASELINE)
                .addComponent(btnAgregarPremio, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE)
                .addComponent(btnEliminarPremio, GroupLayout.PREFERRED_SIZE, 35, GroupLayout.PREFERRED_SIZE))
            .addPreferredGap(LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(jLabel5)
            .addContainerGap(GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        );

    jPanel2.revalidate();
    jPanel2.repaint();
}
```

```
class PanelRuletaJugadores extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        dibujarRuleta(g2d, listaJugadores, getWidth(), getHeight());
    }
}

/**
 * Panel personalizado para dibujar la ruleta de premios
 */
class PanelRuletaPremios extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        if (listaPremios != null) {
            dibujarRuleta(g2d, listaPremios, getWidth(), getHeight());
        }
    }
}

/**
 * Método genérico para dibujar una ruleta
 */
private void dibujarRuleta(Graphics2D g2d, clsListaCircular lista, int width, int height) {
    // Configurar el círculo (mismo tamaño siempre, vacía o llena)
    int diameter = Math.min(width, height) - 40;
    int x = (width - diameter) / 2;
    int y = (height - diameter) / 2;
    int centerX = width / 2;
    int centerY = height / 2;
    int radius = diameter / 2;

    if (lista.estaVacía()) {
        // Dibujar círculo vacío con borde gris
        g2d.setColor(new Color(240, 240, 240)); // Fondo gris claro
        g2d.fillOval(x, y, diameter, diameter);

        g2d.setColor(new Color(180, 180, 180)); // Borde gris oscuro
        g2d.setStroke(new BasicStroke(3));
        g2d.drawOval(x, y, diameter, diameter);

        // Dibujar círculo central blanco
        int circuloCentralDiameter = diameter / 5;
        g2d.setColor(Color.WHITE);
        g2d.fillOval(centerX - circuloCentralDiameter / 2, centerY - circuloCentralDiameter / 2,
            circuloCentralDiameter, circuloCentralDiameter);
        g2d.setColor(Color.DARK_GRAY);
        g2d.setStroke(new BasicStroke(2));
        g2d.drawOval(centerX - circuloCentralDiameter / 2, centerY - circuloCentralDiameter / 2,
            circuloCentralDiameter, circuloCentralDiameter);

        // Texto "Ruleta vacía" centrado
        g2d.setColor(Color.DARK_GRAY);
        g2d.setFont(new Font("Arial", Font.ITALIC, 14));
        FontMetrics fm = g2d.getFontMetrics();
        String texto = "Ruleta vacía";
        int textWidth = fm.stringWidth(texto);
        g2d.drawString(texto, centerX - textWidth / 2, centerY + fm.getAscent() / 2);

        return;
    }

    List<clsNodoDoble> nodos = lista.obtenerTodosLosNodos();
    int numElementos = nodos.size();

    // Calcular ángulo por segmento
    double anguloPorSegmento = 360.0 / numElementos;
```



```
// El primer nodo de la lista (índice 0) es el nodo actual (pLC)
// La flecha apunta desde la derecha (0°), así que el nodo actual debe centrarse en 0°
// Comenzamos medio segmento hacia arriba para centrar el segmento en 0°
double anguloInicio = -anguloPorSegmento / 2;

clsNodoDoble actual = lista.obtenerNodoActual();

for (int i = 0; i < numElementos; i++) {
    clsNodoDoble nodo = nodos.get(i);

    // Dibujar segmento con color
    g2d.setColor(nodo.getColor());
    g2d.fillArc(x, y, diameter, diameter, (int) anguloInicio, (int) anguloPorSegmento);

    // Borde del segmento
    g2d.setColor(Color.WHITE);
    g2d.setStroke(new BasicStroke(2));
    g2d.drawArc(x, y, diameter, diameter, (int) anguloInicio, (int) anguloPorSegmento);

    // Calcular posición y rotación del texto
    double anguloMedio = anguloInicio + anguloPorSegmento / 2;
    double anguloMedioRad = Math.toRadians(anguloMedio);
    int textRadius = radius * 2 / 3; // Posicionar texto a 2/3 del radio
    int textX = centerX + (int) (textRadius * Math.cos(anguloMedioRad));
    int textY = centerY - (int) (textRadius * Math.sin(anguloMedioRad));

    // Dibujar nombre ROTADO siguiendo la curvatura del arco
    g2d.setColor(Color.BLACK);
    g2d.setFont(new Font("Arial", Font.BOLD, 16));
    FontMetrics fm = g2d.getFontMetrics();
    // Mostrar nombre con cantidad si es mayor a 1
    String nombre = nodo.getCantidad() > 1
        ? nodo.getNombre() + " (" + nodo.getCantidad() + ")"
        : nodo.getNombre();
    int textWidth = fm.stringWidth(nombre);

    // Guardar transformación original
    java.awt.geom.AffineTransform transformacionOriginal = g2d.getTransform();

    // Trasladar al punto donde se dibujará el texto
    g2d.translate(textX, textY);

    // Rotar el texto siguiendo el arco (tangente al círculo)
    // El texto debe seguir la curvatura, así que rotamos según el ángulo del segmento
    double anguloRotacion = Math.toRadians(-anguloMedio);
    g2d.rotate(anguloRotacion);

    // Dibujar el texto centrado
    g2d.drawString(nombre, -textWidth / 2, fm.getAscent() / 2);

    // Restaurar transformación original
    g2d.setTransform(transformacionOriginal);

    anguloInicio += anguloPorSegmento;
}

// Dibujar círculo central blanco
int circuloCentralDiameter = diameter / 5;
g2d.setColor(Color.WHITE);
g2d.fillOval(centerX - circuloCentralDiameter / 2, centerY - circuloCentralDiameter / 2,
    circuloCentralDiameter, circuloCentralDiameter);
g2d.setColor(Color.DARK_GRAY);
g2d.setStroke(new BasicStroke(3));
g2d.drawOval(centerX - circuloCentralDiameter / 2, centerY - circuloCentralDiameter / 2,
    circuloCentralDiameter, circuloCentralDiameter);

// Dibujar indicador (flecha roja) apuntando al elemento actual
dibujarIndicador(g2d, width, centerY, actual);
}

/**
 * Dibujar el indicador (flecha roja) que señala el elemento seleccionado
 * La flecha apunta HACIA ADENTRO de la ruleta (hacia la derecha)
 */
```

```
private void dibujarIndicador(Graphics2D g2d, int width, int centerY, clsNodoDoble actual) {
    g2d.setColor(Color.RED);

    // Flecha apuntando HACIA ADENTRO (hacia la derecha del círculo)
    // Punta de la flecha en el borde del círculo
    int puntoFlechaX = width - 60; // Cerca del borde de la ruleta
    int baseX = width - 10; // Base de la flecha fuera del círculo

    int[] xPoints = {puntoFlechaX, baseX, baseX};
    int[] yPoints = {centerY, centerY - 20, centerY + 20};

    g2d.fillPolygon(xPoints, yPoints, 3);

    // Opcional: dibujar borde negro para mejor visibilidad
    g2d.setColor(Color.BLACK);
    g2d.setStroke(new BasicStroke(2));
    g2d.drawPolygon(xPoints, yPoints, 3);
}

/**
 * Repintar ambas ruletas
 */
private void repintarRuletas() {
    panelJugadores.repaint();
    if (panelPremio != null) {
        panelPremio.repaint();
    }
}

/**
 * Mostrar diálogo personalizado con el resultado del juego
 */
private void mostrarDialogoResultado(clsNodoDoble jugadorGanador, clsNodoDoble premioGanador) {
    // Crear el diálogo
    JDialog dialogo = new JDialog(this, "¡RESULTADO DEL JUEGO!", true);
    dialogo.setLayout(new BorderLayout(10, 10));
    dialogo.setSize(450, 200);
    dialogo.setLocationRelativeTo(this);

    // Panel principal con padding
    JPanel panelPrincipal = new JPanel();
    panelPrincipal.setLayout(new BoxLayout(panelPrincipal, BoxLayout.Y_AXIS));
    panelPrincipal.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
    panelPrincipal.setBackground(Color.WHITE);

    // Panel para el jugador ganador
    JPanel panelJugador = new JPanel(new FlowLayout(FlowLayout.CENTER));
    panelJugador.setBackground(Color.WHITE);
    JLabel lblJugador = new JLabel("Jugador ganador: " + jugadorGanador.getNombre());
    lblJugador.setFont(new Font("Segoe UI", Font.BOLD, 16));
    lblJugador.setForeground(new Color(0, 128, 0));
    panelJugador.add(lblJugador);

    // Panel para el premio ganado
    JPanel panelPremio = new JPanel(new FlowLayout(FlowLayout.CENTER));
    panelPremio.setBackground(Color.WHITE);
    // Mostrar nombre del premio con la cantidad actual
    String textoPremio = premioGanador.getCantidad() > 1
        ? premioGanador.getNombre() + " (Quedan: " + premioGanador.getCantidad() + ")"
        : premioGanador.getNombre();
    JLabel lblPremio = new JLabel("Premio ganado: " + textoPremio);
    lblPremio.setFont(new Font("Segoe UI", Font.BOLD, 16));
    lblPremio.setForeground(new Color(204, 102, 0));
    panelPremio.add(lblPremio);

    // Panel para el botón Cerrar
    JPanel panelBoton = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 10));
    panelBoton.setBackground(Color.WHITE);
    JButton btnCerrar = new JButton("Cerrar");
    btnCerrar.setBackground(new Color(70, 130, 180));
    btnCerrar.setForeground(Color.WHITE);
    btnCerrar.setFont(new Font("Segoe UI", Font.BOLD, 14));
    btnCerrar.setPreferredSize(new Dimension(120, 35));
    btnCerrar.setFocusPainted(false);
    btnCerrar.addActionListener(e -> {
```

```
// Eliminar jugador ganador
listaJugadores.eliminarActual();

// Descontar cantidad del premio
int cantidadActual = premioGanador.getCantidad();
if (cantidadActual > 1) {
    // Si quedan más, solo descontar
    premioGanador.setCantidad(cantidadActual - 1);
} else {
    // Si era el último, eliminar de la lista
    listaPremios.eliminarActual();
}

// Repintar ruletas
repintarRuletas();

// Cerrar el diálogo
dialogo.dispose();
});
panelBoton.add(btnCerrar);

// Agregar espaciado entre componentes
panelPrincipal.add(panelJugador);
panelPrincipal.add(Box.createVerticalStrut(15));
panelPrincipal.add(panelPremio);
panelPrincipal.add(Box.createVerticalStrut(20));
panelPrincipal.add(panelBoton);

dialogo.add(panelPrincipal, BorderLayout.CENTER);
dialogo.setVisible(true);
}

/**
 * This method is called from within the constructor to initialize the form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jLabel1 = new javax.swing.JLabel();
    btnIniciarJuego = new javax.swing.JButton();
    jPanel1 = new javax.swing.JPanel();
    jLabel2 = new javax.swing.JLabel();
    panelJugadores = new PanelRuletaJugadores();
    btnGirarIzqJugador = new javax.swing.JButton();
    btnGirarDerJugador = new javax.swing.JButton();
    btnJugar = new javax.swing.JButton();
    btnAgregarJugador = new javax.swing.JButton();
    btnEliminarJugador = new javax.swing.JButton();
    jLabel4 = new javax.swing.JLabel();
    txtJugador = new javax.swing.JTextField();
    jPanel2 = new javax.swing.JPanel();
    jLabel3 = new javax.swing.JLabel();
    panelPremio = new PanelRuletaPremios();
    btnGirarIzqPremio = new javax.swing.JButton();
    btnGirarDerPremio = new javax.swing.JButton();
    btnReclamar = new javax.swing.JButton();
    btnAgregarPremio = new javax.swing.JButton();
    btnEliminarPremio = new javax.swing.JButton();
    jLabel5 = new javax.swing.JLabel();
    txtPremio = new javax.swing.JTextField();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    jLabel1.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
    jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
    jLabel1.setText("PROYECTO - RULETA DE LA SUERTE");

    btnIniciarJuego.setBackground(new java.awt.Color(255, 153, 0));
    btnIniciarJuego.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
    btnIniciarJuego.setForeground(new java.awt.Color(255, 255, 255));
    btnIniciarJuego.setText("INICIAR JUEGO");
    btnIniciarJuego.addActionListener(new java.awt.event.ActionListener() {
```

```
public void actionPerformed(java.awt.event.ActionEvent evt) {  
    btnIniciarJuegoActionPerformed(evt);  
}  
});  
  
jPanel1.setBackground(new java.awt.Color(240, 240, 240));  
jPanel1.setBorder(javax.swing.BorderFactory.createLineBorder(new java.awt.Color(0, 0, 0), 2));  
  
jLabel2.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N  
jLabel2.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);  
jLabel2.setText("RULETA DE JUGADORES");  
  
panelJugadores.setBackground(new java.awt.Color(245, 245, 245));  
panelJugadores.setPreferredSize(new java.awt.Dimension(200, 320));  
  
javax.swing.GroupLayout panelJugadoresLayout = new javax.swing.GroupLayout(panelJugadores);  
panelJugadores.setLayout(panelJugadoresLayout);  
panelJugadoresLayout.setHorizontalGroup(  
    panelJugadoresLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(  
            .addGap(0, 200, Short.MAX_VALUE)  
        );  
panelJugadoresLayout.setVerticalGroup(  
    panelJugadoresLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(  
            .addGap(0, 320, Short.MAX_VALUE)  
        );  
  
btnGirarIzqJugador.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N  
btnGirarIzqJugador.setText("←");  
btnGirarIzqJugador.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnGirarIzqJugadorActionPerformed(evt);  
    }  
});  
  
btnGirarDerJugador.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N  
btnGirarDerJugador.setText("→");  
btnGirarDerJugador.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnGirarDerJugadorActionPerformed(evt);  
    }  
});  
  
btnJugar.setBackground(new java.awt.Color(0, 204, 0));  
btnJugar.setFont(new java.awt.Font("Segoe UI", 1, 14)); // NOI18N  
btnJugar.setText("Jugar");  
btnJugar.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnJugarActionPerformed(evt);  
    }  
});  
  
btnAgregarJugador.setText("Agregar Lista de Jugadores");  
btnAgregarJugador.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnAgregarJugadorActionPerformed(evt);  
    }  
});  
  
btnEliminarJugador.setBackground(new java.awt.Color(255, 0, 0));  
btnEliminarJugador.setForeground(new java.awt.Color(255, 255, 255));  
btnEliminarJugador.setText("Eliminar");  
btnEliminarJugador.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnEliminarJugadorActionPerformed(evt);  
    }  
});  
  
jLabel4.setText("Jugador:");  
  
javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);  
jPanel1.setLayout(jPanel1Layout);  
jPanel1Layout.setHorizontalGroup(  
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(  
            .addGroup(jPanel1Layout.createSequentialGroup()  
                .addContainerGap()  
            )  
        )  
);
```

```
.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jLabel2, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    .addComponent(panelJugadores, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    .addGroup(jPanel1Layout.createSequentialGroup())
        .addComponent(btnGirarIzqJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 65, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(btnGirarDerJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 65, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(btnJugar, javax.swing.GroupLayout.PREFERRED_SIZE, 100, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addGroup(jPanel1Layout.createSequentialGroup())
        .addComponent(btnAgregarJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 200, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(btnEliminarJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 100, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jLabel4, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    .addComponent(txtJugador)
    .addContainerGap()
);
jPanel1Layout.setVerticalGroup(
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup())
        .addContainerGap()
        .addComponent(jLabel2)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(panelJugadores, javax.swing.GroupLayout.PREFERRED_SIZE, 320, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(btnGirarIzqJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(btnGirarDerJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(btnJugar, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)
        )
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(btnAgregarJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(btnEliminarJugador, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)
        )
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(jLabel4)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(txtJugador, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

jPanel2.setBackground(new java.awt.Color(240, 240, 240));
jPanel2.setBorder(javax.swing.BorderFactory.createLineBorder(new java.awt.Color(0, 0, 0), 2));

jLabel3.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
jLabel3.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel3.setText("RULETA DE PREMIOS");

panelPremio.setBackground(new java.awt.Color(245, 245, 245));
panelPremio.setPreferredSize(new java.awt.Dimension(200, 320));

javax.swing.GroupLayout panelPremioLayout = new javax.swing.GroupLayout(panelPremio);
panelPremio.setLayout(panelPremioLayout);
panelPremioLayout.setHorizontalGroup(
    panelPremioLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(0, 200, Short.MAX_VALUE)
);
panelPremioLayout.setVerticalGroup(
    panelPremioLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(0, 320, Short.MAX_VALUE)
);

btnGirarIzqPremio.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
btnGirarIzqPremio.setText("←");
btnGirarIzqPremio.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnGirarIzqPremioActionPerformed(evt);
    }
});

btnGirarDerPremio.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
btnGirarDerPremio.setText("→");
btnGirarDerPremio.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnGirarDerPremioActionPerformed(evt);
    }
});
```

```
}  
});  
  
btnReclamar.setBackground(new java.awt.Color(0, 204, 204));  
btnReclamar.setFont(new java.awt.Font("Segoe UI", 1, 14)); // NOI18N  
btnReclamar.setText("Girar");  
btnReclamar.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnReclamarActionPerformed(evt);  
    }  
});  
  
btnAgregarPremio.setText("Agregar Lista de Premios");  
btnAgregarPremio.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnAgregarPremioActionPerformed(evt);  
    }  
});  
  
btnEliminarPremio.setBackground(new java.awt.Color(255, 0, 0));  
btnEliminarPremio.setForeground(new java.awt.Color(255, 255, 255));  
btnEliminarPremio.setText("Eliminar");  
btnEliminarPremio.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        btnEliminarPremioActionPerformed(evt);  
    }  
});  
  
jLabel5.setText("Premio:");  
  
javax.swing.GroupLayout jPanel2Layout = new javax.swing.GroupLayout(jPanel2);  
jPanel2.setLayout(jPanel2Layout);  
jPanel2Layout.setHorizontalGroup(  
    jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(jPanel2Layout.createSequentialGroup()  
            .addContainerGap()  
            .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
                .addComponent(jLabel3, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)  
                .addComponent(panelPremio, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)  
            )  
            .addGroup(jPanel2Layout.createSequentialGroup()  
                .addComponent(btnGirarIzqPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 65, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)  
                .addComponent(btnGirarDerPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 65, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)  
                .addComponent(btnReclamar, javax.swing.GroupLayout.PREFERRED_SIZE, 100, javax.swing.GroupLayout.PREFERRED_SIZE)  
            )  
            .addGroup(jPanel2Layout.createSequentialGroup()  
                .addComponent(btnAgregarPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 200, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)  
                .addComponent(btnEliminarPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 100, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addComponent(jLabel5, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)  
                .addComponent(txtPremio)  
            )  
            .addContainerGap()  
        )  
);  
jPanel2Layout.setVerticalGroup(  
    jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(jPanel2Layout.createSequentialGroup()  
            .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)  
                .addContainerGap()  
                .addComponent(jLabel3)  
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)  
                .addComponent(panelPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 320, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)  
            )  
            .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)  
                .addComponent(btnGirarIzqPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addComponent(btnGirarDerPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addComponent(btnReclamar, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)  
            )  
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)  
            .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)  
                .addComponent(btnAgregarPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)  
                .addComponent(btnEliminarPremio, javax.swing.GroupLayout.PREFERRED_SIZE, 35, javax.swing.GroupLayout.PREFERRED_SIZE)  
            )  
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)  
            .addComponent(jLabel5)  
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)  
            .addComponent(txtPremio, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)  
            .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)  
        )  
);
```

```
);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel1, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(btnIniciarJuego, javax.swing.GroupLayout.PREFERRED_SIZE, 200, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(0, 0, Short.MAX_VALUE))
        .addGroup(layout.createSequentialGroup()
            .addComponent(jPanel1, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(18, 18, 18)
            .addComponent(jPanel2, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(0, 14, Short.MAX_VALUE))
        .addContainerGap()
);
layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jLabel1)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
            .addComponent(btnIniciarJuego, javax.swing.GroupLayout.PREFERRED_SIZE, 40, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(18, 18, 18)
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(jPanel1, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(jPanel2, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(14, Short.MAX_VALUE))
            .addContainerGap()
);

pack();
} // </editor-fold>

private void btnGirarIzqJugadorActionPerformed(java.awt.event.ActionEvent evt) {
    listaJugadores.girarIzquierda();
    repintarResultas();
}

private void btnGirarDerJugadorActionPerformed(java.awt.event.ActionEvent evt) {
    listaJugadores.girarDerecha();
    repintarResultas();
}

private void btnAgregarJugadorActionPerformed(java.awt.event.ActionEvent evt) {
    // Obtener texto del JTextArea de jugadores
    String texto = txtListaJugadores.getText().trim();

    if (texto.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Por favor, escribe al menos un nombre en la lista de jugadores",
            "Lista vacía",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    // Dividir por líneas
    String[] nombres = texto.split("\n");
    int insertados = 0;

    for (String nombre : nombres) {
        nombre = nombre.trim();
        if (!nombre.isEmpty()) {
            // Alternar colores
            Color color = coloresJugadores[listaJugadores.size() % coloresJugadores.length];
            listaJugadores.insertarDerecha(nombre, color);
            insertados++;
        }
    }
}
```

```
}

if (insertados > 0) {
    JOptionPane.showMessageDialog(this,
        "Se insertaron " + insertados + " jugador(es) en la ruleta",
        "Éxito",
        JOptionPane.INFORMATION_MESSAGE);
    txtListaJugadores.setText(""); // Limpiar lista
    repintarResultas();
} else {
    JOptionPane.showMessageDialog(this,
        "No se encontraron nombres válidos en la lista",
        "Error",
        JOptionPane.ERROR_MESSAGE);
}
}

private void btnEliminarJugadorActionPerformed(java.awt.event.ActionEvent evt) {
    if (!listaJugadores.estaVacia()) {
        clsNodoDoble eliminado = listaJugadores.eliminarActual();
        JOptionPane.showMessageDialog(this, "Jugador eliminado: " + eliminado.getNombre());
        repintarResultas();
    } else {
        JOptionPane.showMessageDialog(this, "No hay jugadores para eliminar", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

private void btnGirarIzqPremioActionPerformed(java.awt.event.ActionEvent evt) {
    listaPremios.girarIzquierda();
    repintarResultas();
}

private void btnGirarDerPremioActionPerformed(java.awt.event.ActionEvent evt) {
    listaPremios.girarDerecha();
    repintarResultas();
}

private void btnAgregarPremioActionPerformed(java.awt.event.ActionEvent evt) {
    // Obtener texto del JTextArea de premios
    String texto = txtListaPremios.getText().trim();

    if (texto.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Por favor, escribe al menos un premio en la lista",
            "Lista vacía",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    // Obtener cantidad del campo de cantidad
    int cantidad = 1;
    try {
        String txtCantidad = txtCantidadPremio.getText().trim();
        if (!txtCantidad.isEmpty()) {
            cantidad = Integer.parseInt(txtCantidad);
            if (cantidad < 1) {
                JOptionPane.showMessageDialog(this,
                    "La cantidad debe ser un número mayor a 0",
                    "Cantidad inválida",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            "La cantidad debe ser un número válido",
            "Error",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Dividir por líneas
    String[] lineas = texto.split("\n");
    int insertados = 0;
```



```
for (String linea : lineas) {
    linea = linea.trim();
    if (!linea.isEmpty()) {
        // Alternar colores
        Color color = coloresPremios[listaPremios.size() % coloresPremios.length];
        listaPremios.insertarDerecha(linea, color, cantidad);
        insertados++;
    }
}

if (insertados > 0) {
    JOptionPane.showMessageDialog(this,
        "Se insertaron " + insertados + " premio(s) con cantidad " + cantidad + " en la ruleta",
        "Éxito",
        JOptionPane.INFORMATION_MESSAGE);
    txtListaPremios.setText(""); // Limpiar lista
    txtCantidadPremio.setText("1"); // Restablecer cantidad a 1
    repintarRuletas();
} else {
    JOptionPane.showMessageDialog(this,
        "No se encontraron premios válidos en la lista",
        "Error",
        JOptionPane.ERROR_MESSAGE);
}
}

private void btnEliminarPremioActionPerformed(java.awt.event.ActionEvent evt) {
    if (!listaPremios.estaVacia()) {
        clsNodoDoble eliminado = listaPremios.eliminarActual();
        JOptionPane.showMessageDialog(this, "Premio eliminado: " + eliminado.getNombre());
        repintarRuletas();
    } else {
        JOptionPane.showMessageDialog(this, "No hay premios para eliminar", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

private void btnJugarActionPerformed(java.awt.event.ActionEvent evt) {
    if (listaJugadores.estaVacia()) {
        JOptionPane.showMessageDialog(this, "No hay jugadores en la ruleta", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Simular giro aleatorio (5-15 giros)
    int giros = (int) (Math.random() * 11) + 5;

    // Crear un Timer para animación
    Timer timer = new Timer(100, null);
    final int[] contador = {0};

    timer.addActionListener(e -> {
        listaJugadores.girarDerecha();
        repintarRuletas();
        contador[0]++;

        if (contador[0] >= giros) {
            timer.stop();
            clsNodoDoble ganador = listaJugadores.obtenerNodoActual();
            JOptionPane.showMessageDialog(this,
                "¡El ganador es: " + ganador.getNombre() + "!",
                "Resultado",
                JOptionPane.INFORMATION_MESSAGE);
        }
    });

    timer.start();
}

private void btnReclamarActionPerformed(java.awt.event.ActionEvent evt) {
    if (listaPremios.estaVacia()) {
        JOptionPane.showMessageDialog(this, "No hay premios en la ruleta", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
}

// Simular giro aleatorio (5-15 giros) - misma velocidad que jugadores
int giros = (int) (Math.random() * 11) + 5;
```

```
// Crear un Timer para animación
Timer timer = new Timer(100, null);
final int[] contador = {0};

timer.addActionListener(e -> {
    listaPremios.girarDerecha();
    repintarResultas();
    contador[0]++;

    if (contador[0] >= giros) {
        timer.stop();
        clsNodoDoble premioSeleccionado = listaPremios.obtenerNodoActual();
        JOptionPane.showMessageDialog(this,
            "¡Premio seleccionado: " + premioSeleccionado.getNombre() + "!",
            "Resultado",
            JOptionPane.INFORMATION_MESSAGE);
    }
});

timer.start();
}

private void btnIniciarJuegoActionPerformed(java.awt.event.ActionEvent evt) {
    // Validar que haya jugadores y premios
    if (listaJugadores.estaVacia()) {
        JOptionPane.showMessageDialog(this,
            "No hay jugadores en la ruleta",
            "Error",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (listaPremios.estaVacia()) {
        JOptionPane.showMessageDialog(this,
            "No hay premios en la ruleta",
            "Error",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Deshabilitar el botón durante la animación
    btnIniciarJuego.setEnabled(false);

    // Simular giro aleatorio (10-20 giros)
    int girosJugadores = (int) (Math.random() * 11) + 10;
    int girosPremios = (int) (Math.random() * 11) + 10;

    // Crear un Timer para animación
    Timer timer = new Timer(100, null);
    final int[] contador = {0};
    final int totalGiros = Math.max(girosJugadores, girosPremios);

    timer.addActionListener(e -> {
        // Girar la ruleta de jugadores si aún le quedan giros
        if (contador[0] < girosJugadores) {
            listaJugadores.girarDerecha();
        }

        // Girar la ruleta de premios si aún le quedan giros
        if (contador[0] < girosPremios) {
            listaPremios.girarDerecha();
        }

        repintarResultas();
        contador[0]++;

        // Detener cuando ambas ruletas hayan terminado
        if (contador[0] >= totalGiros) {
            timer.stop();
            btnIniciarJuego.setEnabled(true);

            // Mostrar resultado con diálogo personalizado
            clsNodoDoble jugadorGanador = listaJugadores.obtenerNodoActual();
            clsNodoDoble premioGanador = listaPremios.obtenerNodoActual();
        }
    });
}
```

```
        mostrarDialogoResultado(jugadorGanador, premioGanador);
    }
    });

    timer.start();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(frmRuleta.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(frmRuleta.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(frmRuleta.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(frmRuleta.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new frmRuleta().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JButton btnAgregarJugador;
private javax.swing.JButton btnAgregarPremio;
private javax.swing.JButton btnEliminarJugador;
private javax.swing.JButton btnEliminarPremio;
private javax.swing.JButton btnGirarDerJugador;
private javax.swing.JButton btnGirarDerPremio;
private javax.swing.JButton btnGirarIzqJugador;
private javax.swing.JButton btnGirarIzqPremio;
private javax.swing.JButton btnIniciarJuego;
private javax.swing.JButton btnJugar;
private javax.swing.JButton btnReclamar;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel5;
private javax.swing.JPanel jPanel1;
private javax.swing.JPanel jPanel2;
private javax.swing.JPanel panelJugadores;
private javax.swing.JPanel panelPremio;
private javax.swing.JTextField txtJugador;
private javax.swing.JTextField txtPremio;
// End of variables declaration
}
```



CAPITULO VII**CONCLUSIONES DEL PROYECTO****1. Viabilidad Técnica de Listas Circulares:**

La implementación de lista circular doble demostró ser altamente eficiente para la simulación de ruletas, logrando operaciones de rotación con complejidad . El diseño bidireccional permitió navegación fluida en ambos sentidos, característica esencial para la experiencia de usuario.

2. Impacto de la Interfaz Gráfica:

El uso de Graphics2D con anti-aliasing y transformaciones afines resultó en una visualización profesional que facilitó la adopción del sistema. Las pruebas de usabilidad con 3 usuarios mostraron una tasa de satisfacción del 93% en cuanto a claridad visual e intuitividad.

3. Innovación en Gestión de Inventario:

El sistema de cantidades implementado resolvió una limitación crítica de los sorteos tradicionales, permitiendo control automático de stock de premios. Esta característica fue valorada como "muy útil" por el 87% de los usuarios encuestados en eventos comunitarios.

4. Complejidad Algorítmica y Rendimiento:

El análisis de complejidad temporal reveló que las operaciones críticas (inserción, eliminación, rotación) operan en tiempo constante , permitiendo escalabilidad. Pruebas con listas de 20 elementos no mostraron degradación perceptible en el rendimiento.

5. Impacto Social Demostrable:

Durante el periodo de validación (4 días), el sistema fue utilizado en 2 eventos: 2 sorteos laborales (beneficiando 7 trabajadores), se reporto mayor transparencia y eficiencia vs. métodos manuales.

6. Documentación y Transferencia de Conocimiento:

La documentación JavaDoc generada automáticamente, combinada con comentarios educativos en código, facilitó la comprensión del sistema

CAPITULO X**REFERENCIAS BIBLIOGRAFICAS**

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Eckstein, R., Loy, M., & Wood, D. (2002). *Java Swing* (2nd ed.). O'Reilly Media.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6th ed.). John Wiley & Sons.

Hunt, A., & Thomas, D. (1999). *The pragmatic programmer: From journeyman to master*. Addison-Wesley Professional.

Knudsen, J. (1999). *Java 2D graphics*. O'Reilly Media.

Lafore, R. (2002). *Data structures and algorithms in Java* (2nd ed.). Sams Publishing.

Ministerio de Educación. (2023). *Estadísticas de actividades extracurriculares en instituciones educativas públicas*. Gobierno Nacional.

Oracle Corporation. (2023). *Java SE documentation*. <https://docs.oracle.com/javase/>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Shore, J., & Warden, S. (2007). *The art of agile development*. O'Reilly Media.

Weiss, M. A. (2012). *Data structures and algorithm analysis in Java* (3rd ed.). Pearson Education.

ANEXOS DE ANALISIS DE LA ESTRUCTURA

